

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN MÔN HỌC: CHUYÊN ĐỀ ASP.NET

HỌC KỲ III, NĂM HỌC 2024-2026

**XÂY DỰNG WEBSITE ĐẶT LỊCH HẸN KHÁM
BỆNH TRỰC TUYẾN BẰNG ASP.NET MVC**

Giảng viên hướng dẫn:

TS. Đoàn Phước Miên

Sinh viên thực hiện:

Họ tên: Huỳnh Hoàng Vinh

MSSV: 170124660

Lớp: DK24TT80171

Vĩnh Long, năm 2026

[illegible]

(Ký tên và ghi rõ họ tên)

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the entire width of the page. There are no margins, text, or other markings present.

Thành viên hội đồng
(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Trong suốt quá trình học tập và nghiên cứu để hoàn thành đồ án tốt nghiệp này, em đã nhận được rất nhiều sự giúp đỡ, động viên và hướng dẫn tận tình từ quý thầy cô, gia đình và bạn bè.

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy TS. Đoàn Phước Miên — người đã trực tiếp hướng dẫn em thực hiện đề tài. Thầy đã dành nhiều thời gian, công sức để định hướng, góp ý và chỉ bảo cho em từ những bước đầu tiên cho đến khi hoàn thành đồ án. Những nhận xét quý báu của thầy đã giúp em vượt qua các khó khăn về mặt kỹ thuật cũng như cách trình bày một báo cáo khoa học.

Em cũng xin gửi lời cảm ơn đến toàn thể quý thầy cô trong Khoa Công nghệ Thông tin đã truyền đạt cho em những kiến thức nền tảng và chuyên ngành quý báu trong suốt quá trình học tập. Những kiến thức đó là hành trang vững chắc để em có thể tự tin thực hiện đề tài và bước vào con đường nghề nghiệp sau này.

Cuối cùng, em xin gửi lời cảm ơn đến gia đình, người thân và bạn bè đã luôn bên cạnh, động viên và tạo điều kiện thuận lợi nhất để em có thể tập trung học tập và hoàn thành đồ án.

Do thời gian thực hiện và kiến thức còn hạn chế, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô và các bạn để đồ án được hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

CHƯƠNG 1: NGHIÊN CỨU LÝ THUYẾT	2
1.1. Tổng quan về ứng dụng web đặt lịch khám bệnh	2
1.2. Kiến trúc ASP.NET Core MVC	3
1.3. Entity Framework Core và mô hình Code First	3
1.4. Cơ sở dữ liệu SQL Server	4
1.5. Cookie Authentication và phân quyền theo vai trò	5
1.6. Gửi email tự động với MailKit	5
1.7. Bootstrap 5 và xây dựng giao diện responsive	6
1.8. Mô hình phát triển và công cụ hỗ trợ	6
CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU	7
2.1. Mô tả bài toán	7
2.2. Yêu cầu chức năng và phi chức năng	7
2.3. Sơ đồ Use Case (UML)	12
2.4. Sơ đồ lớp (Class Diagram - UML)	13
2.5. Thiết kế cơ sở dữ liệu	14
2.6. Thiết kế kiến trúc hệ thống	22
2.7. Thiết kế giao diện	23
CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU	24
3.1. Giao diện hệ thống	24
3.1. Cách thức cài đặt và triển khai	29
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	30
4.1. Kết luận	30
4.1.1. Kết quả đạt được	30
4.1.2. Đóng góp mới	30
4.1.3. Hạn chế	31
4.2. Hướng phát triển	31

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ Use Case tổng quát của hệ thống	12
Hình 2.2: Class Diagram – UML.....	13
Hình 2.3: Thiết kế cơ sở dữ liệu	14
Hình 3.1: Giao diện trang chủ	24
Hình 3.2: Giao diện trang đăng ký/đăng nhập.....	25
Hình 3.3: Giao diện lịch hẹn của Bác Sĩ.....	26
Hình 3.4: Giao diện kết quả khám của Bác Sĩ	26
Hình 3.5: Giao diện tìm bác sĩ của Bệnh nhân	27
Hình 3.6: Giao diện đặt lịch khám.....	27
Hình 3.7: Giao diện lịch hẹn của bệnh nhân.....	28
Hình 3.7: Giao diện của Admin (quản trị viên).....	28

DANH MỤC BẢNG BIỂU

Bảng 2.1: Các tác nhân và Use Case chính của hệ thống	13
Bảng 2.2: Danh sách các Controller chính và phạm vi truy cập	22
Bảng 2.3: Danh sách các màn hình chính của hệ thống	23

TÓM TẮT ĐỒ ÁN

Đồ án "Xây dựng website đặt lịch hẹn khám bệnh trực tuyến" tập trung giải quyết bài toán quản lý lịch hẹn khám bệnh tại các phòng khám và bệnh viện quy mô vừa và nhỏ, nơi quy trình tiếp nhận lịch hẹn vẫn chủ yếu dựa trên gọi điện thoại hoặc đến trực tiếp, dẫn đến quá tải tại quầy lễ tân, dễ trùng lịch và thiếu công cụ theo dõi minh bạch trạng thái cho cả bệnh nhân lẫn bác sĩ.

Hệ thống được xây dựng trên nền tảng ASP.NET Core MVC 8 kết hợp Entity Framework Core (Code First) và cơ sở dữ liệu SQL Server, cung cấp ba vai trò người dùng tách biệt: Bệnh nhân, Bác sĩ và Quản trị viên. Bệnh nhân có thể đăng ký tài khoản, tra cứu danh sách chuyên khoa và bác sĩ, đặt lịch theo khung giờ rảnh, hủy lịch và xem hồ sơ bệnh án sau khi khám. Bác sĩ xem các lịch hẹn của riêng mình, xác nhận, từ chối hoặc nhập kết quả khám. Quản trị viên quản lý toàn bộ danh mục chuyên khoa, bác sĩ, người dùng, lịch hẹn và cấu hình email gửi đi.

Đồ án đã hoàn thành đầy đủ 21 chức năng nghiệp vụ thuộc bốn nhóm (Khách, Bệnh nhân, Bác sĩ, Admin), thiết kế cơ sở dữ liệu gồm bảy bảng với các ràng buộc toàn vẹn, áp dụng Cookie Authentication phân quyền theo vai trò, tích hợp MailKit gửi email tự động khi đặt lịch/xác nhận/hủy lịch và xây dựng Dashboard thống kê trực quan bằng Chart.js. Hệ thống chạy ổn định trên môi trường local, đáp ứng được mục tiêu nghiên cứu đặt ra ban đầu và sẵn sàng cho việc mở rộng trong các giai đoạn tiếp theo.

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, nhu cầu khám chữa bệnh tại Việt Nam ngày càng tăng cao trong khi quy trình đặt lịch hẹn tại nhiều phòng khám và bệnh viện vẫn chủ yếu dựa trên phương thức thủ công như gọi điện thoại, đến trực tiếp xếp hàng hoặc gửi tin nhắn. Cách làm này dẫn đến quá tải tại quầy lễ tân, dễ xảy ra trùng lịch giữa các bệnh nhân, thông tin lịch hẹn không được lưu trữ tập trung và bệnh nhân khó theo dõi trạng thái lịch của mình. Bên cạnh đó, bác sĩ cũng gặp khó khăn trong việc nắm bắt lịch khám trong ngày khi không có công cụ quản lý lịch hiệu quả. Trong bối cảnh chuyển đổi số y tế đang được Bộ Y tế đẩy mạnh, việc xây dựng một website đặt lịch hẹn khám bệnh trực tuyến là hết sức cần thiết. Đây cũng là lý do em chọn đề tài "Xây dựng website đặt lịch hẹn khám bệnh trực tuyến" làm đề tài đồ án tốt nghiệp, vừa giải quyết được nhu cầu thực tế, vừa giúp em vận dụng kiến thức về lập trình web, cơ sở dữ liệu và phân tích thiết kế hệ thống đã được học.

2. Mục tiêu nghiên cứu

Đồ án hướng đến các mục tiêu cụ thể sau: (1) Phân tích và thiết kế một hệ thống đặt lịch hẹn khám bệnh trực tuyến đáp ứng đầy đủ nghiệp vụ cho ba vai trò người dùng là Bệnh nhân, Bác sĩ và Quản trị viên; (2) Xây dựng website hoàn chỉnh trên nền tảng ASP.NET Core MVC 8 với cơ sở dữ liệu SQL Server, áp dụng Entity Framework Core theo mô hình Code First; (3) Triển khai các chức năng cốt lõi gồm quản lý chuyên khoa, quản lý bác sĩ, đặt lịch hẹn, xác nhận và hủy lịch, nhập kết quả khám và quản trị toàn hệ thống; (4) Tích hợp gửi email tự động bằng MailKit để thông báo trạng thái lịch hẹn cho bệnh nhân; (5) Đảm bảo hệ thống có giao diện thân thiện, responsive trên cả máy tính và điện thoại, có cơ chế phân quyền truy cập rõ ràng và đủ độ ổn định để demo trên môi trường thực tế.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đồ án là quy trình nghiệp vụ đặt lịch hẹn khám bệnh tại phòng khám/bệnh viện quy mô vừa và nhỏ, cùng với các công nghệ phục vụ xây dựng ứng

dụng web hiện đại bao gồm ASP.NET Core MVC, Entity Framework Core, SQL Server, MailKit, Bootstrap 5 và Chart.js. Phạm vi nghiên cứu được giới hạn trong việc xây dựng một website chạy trên môi trường local, phục vụ một cơ sở khám chữa bệnh duy nhất, không bao gồm các chức năng thanh toán trực tuyến, ứng dụng di động riêng biệt hay hệ thống quản lý đa chi nhánh. Dữ liệu mẫu được nạp tự động khi khởi động lần đầu để phục vụ demo, gồm năm chuyên khoa cơ bản (Nội, Ngoại, Sản, Nhi, Mắt), một tài khoản quản trị, một số bác sĩ và bệnh nhân mẫu.

4. Phương pháp nghiên cứu

Đồ án sử dụng kết hợp nhiều phương pháp nghiên cứu: (1) Phương pháp nghiên cứu tài liệu - tìm hiểu lý thuyết về kiến trúc MVC, Entity Framework Core, cơ chế xác thực Cookie Authentication, mô hình gửi mail SMTP qua MailKit và các tài liệu chính thức của Microsoft về ASP.NET Core 8; (2) Phương pháp phân tích - khảo sát quy trình thực tế tại phòng khám, từ đó xác định danh sách yêu cầu chức năng và phi chức năng; (3) Phương pháp thiết kế - sử dụng UML để xây dựng sơ đồ Use Case, sơ đồ lớp, sơ đồ tuần tự và sơ đồ ERD, đồng thời lựa chọn kiến trúc ba tầng MVC cho hệ thống; (4) Phương pháp thực nghiệm - cài đặt mã nguồn, chạy thử trên môi trường local, xây dựng các test case kiểm thử chức năng và đánh giá kết quả; (5) Phương pháp so sánh - đối chiếu các quyết định thiết kế với một số website đặt lịch khám tham khảo để đảm bảo tính phù hợp và đầy đủ.

CHƯƠNG 1: NGHIÊN CỨU LÝ THUYẾT

1.1. Tổng quan về ứng dụng web đặt lịch khám bệnh

Ứng dụng web đặt lịch hẹn khám bệnh trực tuyến là một dạng hệ thống thông tin y tế cho phép bệnh nhân chủ động tìm kiếm bác sĩ, lựa chọn chuyên khoa và đăng ký khung giờ khám phù hợp thông qua trình duyệt web mà không cần đến trực tiếp cơ sở y tế. Hệ thống đóng vai trò trung gian giữa ba bên: bệnh nhân, bác sĩ và quản trị viên của phòng khám, giúp đồng bộ thông tin lịch hẹn và giảm tải khâu lễ tân.

Trên thế giới, mô hình này đã được triển khai rộng rãi qua các nền tảng như Zocdoc

(Mỹ), Doctolib (Pháp) hay Practo (Ấn Độ). Tại Việt Nam, một số nền tảng tiêu biểu có thể kể đến như BookingCare, Medpro, hệ thống đặt lịch của Bệnh viện FV, Vinmec. Đặc điểm chung của các hệ thống này là tích hợp danh sách bác sĩ theo chuyên khoa, lịch trống theo khung giờ, xác nhận lịch qua email hoặc SMS, và lưu trữ lịch sử khám của bệnh nhân.

Đề tài tập trung vào phân tích, thiết kế và xây dựng một website đặt lịch hẹn khám bệnh quy mô phòng khám vừa và nhỏ, kế thừa các nguyên tắc nghiệp vụ phổ biến của các hệ thống đã có, đồng thời điều chỉnh để phù hợp với điều kiện triển khai thực tế của một đơn vị kinh doanh.

1.2. Kiến trúc ASP.NET Core MVC

ASP.NET Core là nền tảng phát triển ứng dụng web mã nguồn mở do Microsoft phát triển, đa nền tảng (Windows, Linux, macOS), kế thừa và viết lại từ ASP.NET truyền thống với hiệu năng cao hơn. Mô hình MVC (Model — View — Controller) trong ASP.NET Core chia ứng dụng thành ba thành phần độc lập: Model chịu trách nhiệm về dữ liệu và logic nghiệp vụ, View hiển thị giao diện cho người dùng thông qua các tệp Razor (.cshtml), và Controller xử lý yêu cầu HTTP, điều phối giữa Model và View.

Sự tách biệt này giúp mã nguồn dễ bảo trì, dễ kiểm thử và cho phép nhiều lập trình viên cùng làm việc trên các tầng khác nhau. Trong đồ án, kiến trúc MVC được áp dụng một cách tự nhiên: thư mục Controllers chứa các lớp xử lý yêu cầu theo chức năng (TaiKhoanController, LichHenController, AdminBacSiController...), thư mục Models chứa các entity ánh xạ với cơ sở dữ liệu, và thư mục Views chứa các trang Razor được tổ chức theo từng controller.

ASP.NET Core sử dụng middleware pipeline để xử lý request: mỗi request đi qua một chuỗi middleware tuần tự gồm Static Files, Routing, Session, Authentication, Authorization và cuối cùng là MVC để định tuyến đến action phù hợp. Mô hình dependency injection được tích hợp sẵn cho phép đăng ký và sử dụng các service như DbContext, EmailService một cách linh hoạt

1.3. Entity Framework Core và mô hình Code First

Entity Framework Core (EF Core) là ORM (Object-Relational Mapper) chính thức của Microsoft trên nền tảng .NET. EF Core cho phép thao tác với cơ sở dữ liệu thông qua các đối tượng C# thay vì viết SQL trực tiếp, giúp rút ngắn thời gian phát triển và giảm thiểu lỗi cú pháp SQL.

Đồ án sử dụng EF Core theo mô hình Code First — lập trình viên định nghĩa các lớp entity C# trước, sau đó EF Core sẽ tự sinh ra schema cơ sở dữ liệu tương ứng thông qua cơ chế Migrations. Cách tiếp cận này phù hợp với các dự án mới, nơi chưa có sẵn cấu trúc cơ sở dữ liệu và lập trình viên muốn kiểm soát toàn bộ quá trình thay đổi schema thông qua mã nguồn.

Các tính năng chính của EF Core được khai thác trong đồ án bao gồm: ánh xạ quan hệ một-một và một-nhiều thông qua Fluent API, ràng buộc index unique để chống trùng dữ liệu, eager loading bằng Include / ThenInclude để tải dữ liệu liên quan trong một truy vấn, và LINQ to Entities để viết truy vấn theo kiểu hướng đối tượng. Cơ chế Migrations cho phép phát triển song song giữa nhiều môi trường và rollback khi cần.

1.4. Cơ sở dữ liệu SQL Server

SQL Server là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) thương mại của Microsoft, hỗ trợ chuẩn ANSI SQL cùng các phần mở rộng T-SQL. Phiên bản SQL Server Express miễn phí cho mục đích phát triển và các ứng dụng quy mô nhỏ là lựa chọn phù hợp cho đồ án tốt nghiệp.

Trong đề tài, SQL Server đảm nhiệm việc lưu trữ toàn bộ dữ liệu nghiệp vụ gồm tài khoản người dùng, danh mục chuyên khoa, hồ sơ bác sĩ, hồ sơ bệnh nhân, lịch hẹn, hồ sơ bệnh án và cấu hình email. Các ràng buộc khóa chính, khóa ngoại, UNIQUE và NOT NULL đảm bảo tính toàn vẹn dữ liệu ở mức cơ sở dữ liệu, độc lập với tầng ứng dụng.

SQL Server được tích hợp tốt với EF Core thông qua provider Microsoft.EntityFrameworkCore.SqlServer. Connection string được khai báo trong appsettings.json, cho phép thay đổi dễ dàng giữa các môi trường mà không cần biên dịch lại ứng dụng.

1.5. Cookie Authentication và phân quyền theo vai trò

ASP.NET Core cung cấp nhiều cơ chế xác thực: Cookie Authentication, JWT Bearer, OpenID Connect... Đối với ứng dụng web truyền thống render phía server như đề án này, Cookie Authentication là lựa chọn phù hợp nhất do đơn giản, an toàn và được tích hợp sẵn vào pipeline.

Khi người dùng đăng nhập thành công, hệ thống tạo một ClaimsPrincipal chứa các claim như NameIdentifier, Name, Email, Role, HoTen, sau đó gọi HttpContext.SignInAsync để mã hóa thông tin này vào cookie. Trong các request tiếp theo, middleware Authentication tự động giải mã cookie và gắn ClaimsPrincipal vào HttpContext.User.

Phân quyền được thực hiện bằng attribute [Authorize(Roles = "Admin")] hoặc [Authorize(Roles = "BacSi,Admin")] đặt trên controller hoặc action. Ngoài ra, đề án còn kiểm tra quyền sở hữu dữ liệu ngay trong action — ví dụ một bác sĩ chỉ có thể thao tác trên những lịch hẹn được gán cho chính mình, ngăn ngừa lỗ hổng truy cập ngang.

1.6. Gửi email tự động với MailKit

MailKit là thư viện gửi và nhận email mã nguồn mở phổ biến trên .NET, được khuyến nghị thay thế cho Smtplib cũ vốn đã bị Microsoft đánh dấu lỗi thời. MailKit hỗ trợ đầy đủ các giao thức SMTP, IMAP, POP3, kèm theo các tính năng nâng cao như SSL/TLS, OAuth2 và gửi mail bất đồng bộ.

Trong đề án, MailKit được đóng gói trong lớp EmailService với ba phương thức chính: GuiMailDatLichAsync (gửi khi bệnh nhân đặt lịch thành công), GuiMailXacNhanLichAsync (gửi khi bác sĩ xác nhận lịch) và GuiMailHuyLichAsync (gửi khi lịch bị hủy). Cấu hình SMTP (server, port, sender email, app password) được lưu trong bảng CauHinhEmail và đọc qua lớp EmailSettingsProvider, cho phép quản trị viên thay đổi cấu hình ngay trên giao diện web mà không cần restart ứng dụng.

Cơ chế bật/tắt gửi mail toàn hệ thống được thiết kế để phục vụ môi trường demo:

khi cờ BatEmail = false hoặc cấu hình SMTP chưa hợp lệ, hệ thống chỉ ghi log [EMAIL-OFF] / [EMAIL-DEV] thay vì gửi mail thật, giúp việc kiểm thử không phụ thuộc vào kết nối SMTP bên ngoài.

1.7. Bootstrap 5 và xây dựng giao diện responsive

Bootstrap 5 là framework CSS-JavaScript front-end phổ biến nhất hiện nay, cung cấp hệ thống lưới 12 cột, các thành phần UI dựng sẵn (navbar, modal, card, form, alert, badge...) và class tiện ích (utility class). Bootstrap 5 đã loại bỏ phụ thuộc jQuery, dùng JavaScript thuần và hỗ trợ tốt các trình duyệt hiện đại.

Đồ án sử dụng Bootstrap 5 cùng Bootstrap Icons để xây dựng giao diện responsive cho cả hai layout: _Layout.cshtml cho người dùng cuối (bệnh nhân, bác sĩ, khách) và _AdminLayout.cshtml cho quản trị viên với thanh sidebar tối. Hệ thống lưới col-md-* được dùng cho các trang danh sách bác sĩ, danh sách chuyên khoa, dashboard và form đặt lịch, đảm bảo hiển thị tốt trên cả desktop, tablet và mobile.

Một số thành phần UI nâng cao được áp dụng gồm modal (xem hồ sơ bệnh án), badge màu (trạng thái lịch hẹn), alert (TempData thông báo sau POST), tab (lọc lịch theo trạng thái phía bác sĩ) và biểu đồ Chart.js (dashboard Admin).

1.8. Mô hình phát triển và công cụ hỗ trợ

Đồ án được phát triển theo mô hình tăng dần — mỗi sprint hoàn thành một nhóm chức năng (quản lý tài khoản, quản lý lịch hẹn, gửi email, dashboard) trước khi chuyển sang nhóm tiếp theo. Cách tiếp cận này phù hợp với phạm vi và thời gian thực hiện đồ án.

Công cụ hỗ trợ chính bao gồm: Visual Studio 2022 / Visual Studio Code (IDE), SQL Server Management Studio (quản trị cơ sở dữ liệu), Git (quản lý mã nguồn), Postman (kiểm thử request HTTP) và trình duyệt Chrome DevTools (debug giao diện). Các script PowerShell tiện ích (chay-web.ps1, cap-nhat-database.ps1, reset-database.ps1) được viết bổ sung để rút ngắn các thao tác lặp lại trong quá trình phát triển và demo.

CHƯƠNG 2: HIỆN THỰC HÓA NGHIÊN CỨU

2.1. Mô tả bài toán

Hệ thống đặt lịch khám bệnh trực tuyến được xây dựng nhằm hỗ trợ phòng khám hoặc bệnh viện quy mô nhỏ trong việc quản lý lịch hẹn của bệnh nhân với bác sĩ. Hiện nay, tại nhiều cơ sở y tế, việc đặt lịch khám vẫn chủ yếu thực hiện qua điện thoại hoặc tại quầy lễ tân, dẫn đến tình trạng quá tải, dễ trùng lịch và khó tra cứu thông tin. Đề tài này giải quyết bài toán đó bằng một website cho phép bệnh nhân tự đặt lịch trực tuyến, bác sĩ chủ động xác nhận và quản lý lịch của mình, đồng thời quản trị viên có công cụ giám sát toàn hệ thống.

Hệ thống được phân chia thành ba vai trò chính: Bệnh nhân thực hiện đăng ký tài khoản, tìm kiếm bác sĩ, đặt lịch và theo dõi trạng thái lịch hẹn của bản thân; Bác sĩ tiếp nhận, xác nhận hoặc từ chối lịch hẹn, đồng thời nhập kết quả khám sau khi tiếp xúc bệnh nhân; Quản trị viên (Admin) quản lý chuyên khoa, danh sách bác sĩ, người dùng, lịch hẹn và cấu hình email thông báo của toàn hệ thống. Bên cạnh đó, một vai trò Khách (chưa đăng nhập) cũng được hỗ trợ để xem thông tin chuyên khoa và bác sĩ trước khi quyết định đăng ký.

Toàn bộ luồng nghiệp vụ của hệ thống xoay quanh đối tượng Lịch hẹn (LichHen) với bốn trạng thái: Chờ xác nhận, Đã xác nhận, Đã khám, Đã hủy. Mỗi lần chuyển trạng thái sẽ kèm theo việc gửi email thông báo cho bệnh nhân (nếu Admin bật chức năng gửi mail). Sau khi lịch chuyển sang trạng thái Đã khám, hệ thống tự động tạo hoặc cập nhật một Hồ sơ bệnh án (HoSoBenhAn) gắn với lịch đó, lưu lại chẩn đoán, đơn thuốc và lời khuyên của bác sĩ.

2.2. Yêu cầu chức năng và phi chức năng

Trên cơ sở khảo sát quy trình thực tế và phân nhóm người dùng, hệ thống được xác định 21 chức năng chính (F01–F21) phân bổ theo bốn nhóm: Khách / Công khai, Bệnh nhân, Bác sĩ và Quản trị viên. Đồng thời tám yêu cầu phi chức năng (NF01–NF08) được đặt ra nhằm đảm bảo chất lượng hệ thống về mặt bảo mật, hiệu năng, khả năng mở rộng và

trải nghiệm người dùng. Chi tiết các yêu cầu này được trình bày trong các bảng dưới đây.

a) Yêu cầu chức năng cho nhóm Khách (chưa đăng nhập):

Mã	Chức năng	Mô tả
F01	Xem trang chủ	Hiển thị thống kê số bác sĩ, chuyên khoa, bệnh nhân và danh sách chuyên khoa/bác sĩ nổi bật.
F02	Xem danh sách chuyên khoa	Liệt kê tất cả chuyên khoa của hệ thống.
F03	Xem chi tiết chuyên khoa	Hiển thị danh sách bác sĩ thuộc chuyên khoa được chọn.
F04	Tìm kiếm bác sĩ	Lọc bác sĩ theo tên, học vị, mô tả và chuyên khoa.
F05	Xem chi tiết bác sĩ	Học vị, kinh nghiệm, giá khám và chuyên khoa của bác sĩ.
F06	Đăng ký tài khoản	Tạo tài khoản bệnh nhân mới với vai trò BenhNhan.
F07	Đăng nhập/Đăng xuất	Xác thực qua Cookie Authentication, phân quyền theo vai trò.

b) Yêu cầu chức năng cho Bệnh nhân:

Mã	Chức năng	Mô tả
F08	Đặt lịch khám	Chọn bác sĩ, ngày khám ($\geq$ ngày mai), khung giờ và nhập lý do khám.
F09	Xem lịch hẹn của tôi	Danh sách lịch hẹn kèm trạng thái, xem hồ sơ bệnh án qua modal.
F10	Hủy lịch hẹn	Chỉ cho phép hủy khi lịch đang Chờ xác nhận hoặc Đã xác nhận.
F11	Nhận email thông báo	Nhận email khi đặt lịch, khi bác sĩ xác nhận và khi lịch bị hủy.

c) Yêu cầu chức năng cho Bác sĩ:

Mã	Chức năng	Mô tả
F12	Xem lịch hẹn đến với mình	Lọc theo trạng thái Chờ xác nhận/Đã xác nhận/Đã khám/Đã hủy.
F13	Xác nhận lịch hẹn	Chuyển trạng thái từ Chờ xác nhận sang Đã xác

		nhận, gửi email cho bệnh nhân.
F14	Từ chối/Hủy lịch	Chuyển trạng thái lịch sang Đã hủy và gửi email thông báo.
F15	Nhập kết quả khám	Tạo hoặc cập nhật Hồ sơ bệnh án, chuyển lịch sang trạng thái Đã khám.

d) Yêu cầu chức năng cho Quản trị viên (Admin):

Mã	Chức năng	Mô tả
F16	Dashboard thống kê	Số liệu tổng quan, biểu đồ Chart.js, Top 5 bác sĩ, 5 lịch hẹn mới nhất.
F17	CRUD Chuyên khoa	Thêm / sửa / xóa chuyên khoa (chặn xóa khi vẫn còn bác sĩ).
F18	CRUD Bác sĩ	Tạo bác sĩ kèm tài khoản, sửa, xóa (chặn nếu còn lịch hẹn).
F19	Quản lý người dùng	Xem, lọc, sửa, khóa và xóa tài khoản.
F20	Quản lý lịch hẹn	Xem toàn bộ lịch, lọc theo

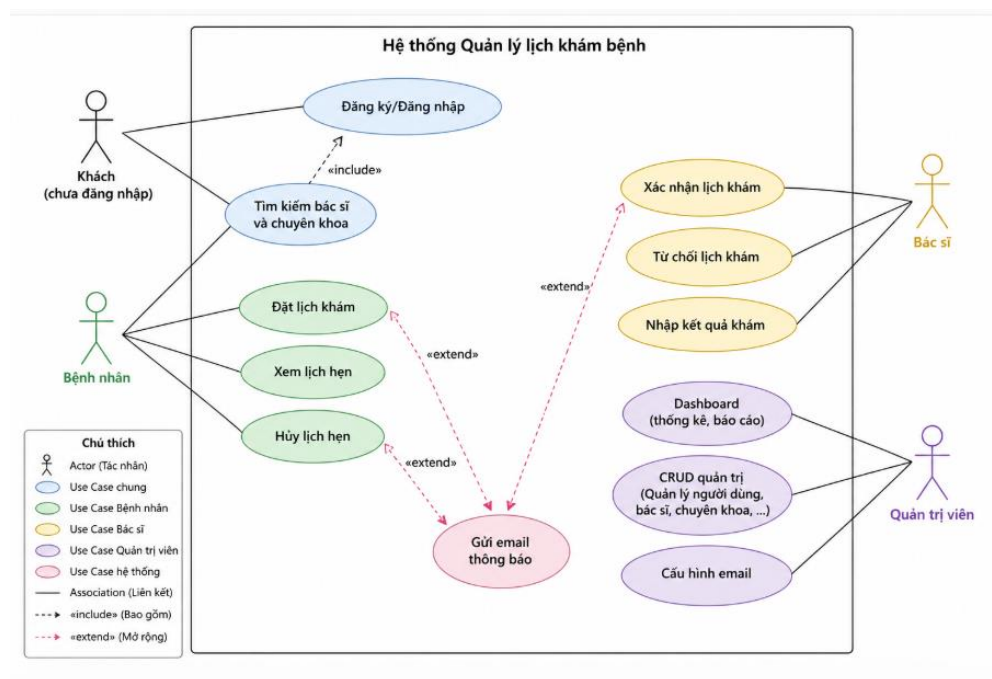
		bác sĩ / trạng thái / ngày, Admin có thể hủy lịch.
F21	Cấu hình Email	Bật / tắt gửi mail, cấu hình SMTP, thực hiện gửi mail thử.

e) Yêu cầu phi chức năng:

Mã	Yêu cầu	Cách đáp ứng
NF01	Bảo mật truy cập	Cookie Authentication kết hợp [Authorize(Roles)], kiểm tra quyền sở hữu lịch hẹn.
NF02	Hiệu năng chấp nhận được	Sử dụng Entity Framework Core, index unique, eager loading có chọn lọc.
NF03	Giao diện thân thiện	Bootstrap 5 responsive, Bootstrap Icons.
NF04	Dễ bảo trì	Kiến trúc MVC 3 tầng, tách lớp Services (Email).
NF05	Khả năng mở rộng CSDL	Entity Framework Core Code First kết hợp Migrations.

NF06	Thông báo email	Gửi mail qua MailKit SMTP, Admin có thể bật / tắt trên giao diện.
NF07	Validation dữ liệu	Data Annotations kết hợp jQuery Validation phía client.
NF08	Triển khai đơn giản	Lệnh dotnet run, tự động migrate và seed dữ liệu khi khởi động.

2.3. Sơ đồ Use Case (UML)

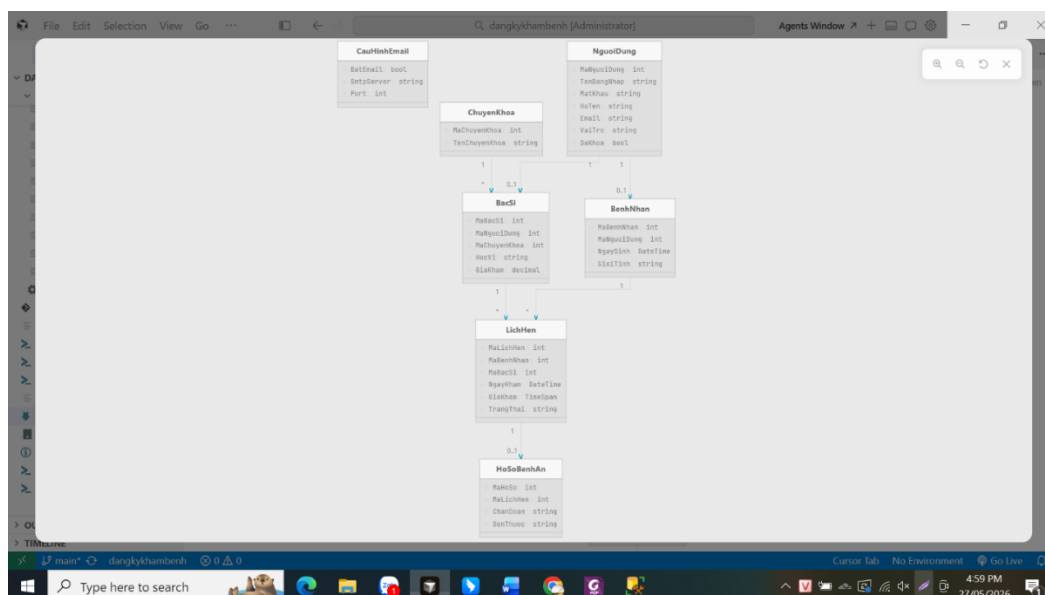


Hình 2.1: Sơ đồ Use Case tổng quát của hệ thống

Bảng 2.1 dưới đây tóm tắt các tác nhân và Use Case chính tương ứng:

Tác nhân	Use Case chính
Khách	Xem trang chủ, danh sách chuyên khoa và bác sĩ; đăng ký, đăng nhập.
Bệnh nhân	Đặt lịch khám, xem và hủy lịch hẹn của mình, nhận email thông báo.
Bác sĩ	Xem lịch hẹn, xác nhận, từ chối và nhập kết quả khám.
Quản trị viên	Dashboard, CRUD chuyên khoa / bác sĩ, quản lý người dùng, quản lý lịch hẹn, cấu hình email.

2.4. Sơ đồ lớp (Class Diagram - UML)

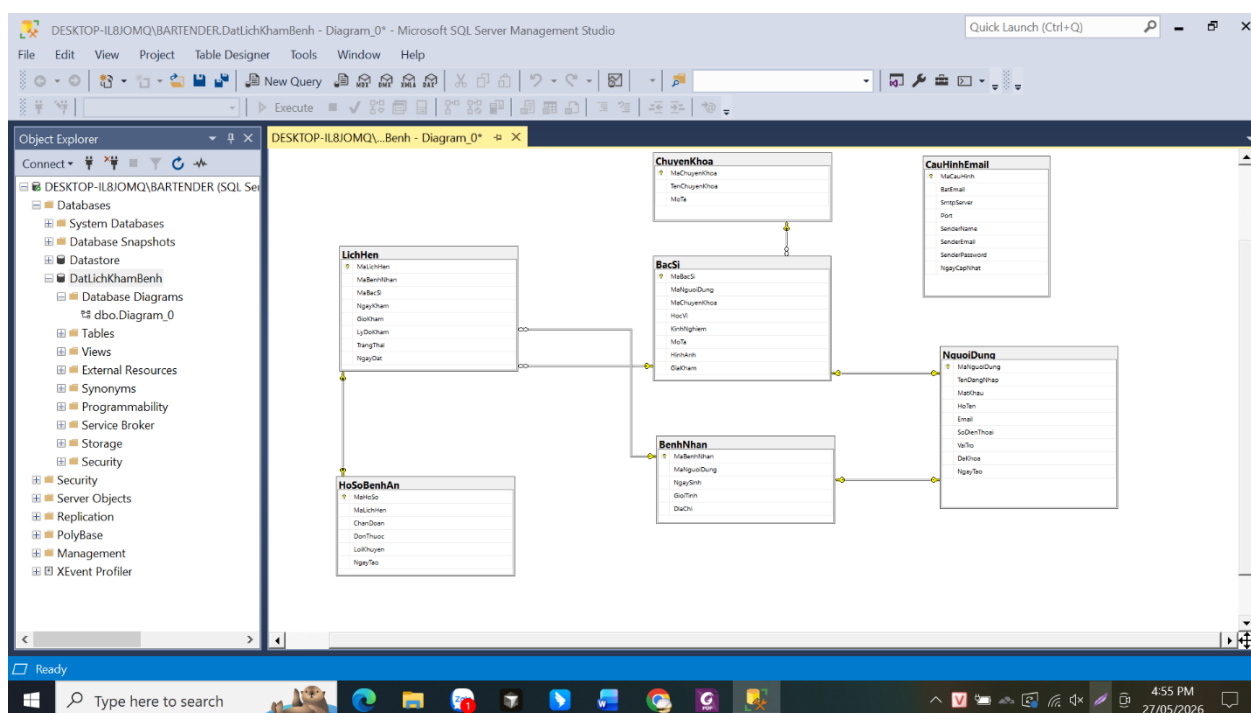


Hình 2.2: Class Diagram - UML

Bên cạnh các lớp thực thể, hệ thống có các lớp dịch vụ (Services) phục vụ logic nghiệp vụ: IEmailService và EmailService chịu trách nhiệm gửi email qua MailKit cho ba sự kiện chính (đặt lịch, xác nhận lịch, hủy lịch); IEmailSettingsProvider và EmailSettingsProvider đọc cấu hình SMTP từ bảng CauHinhEmail; AppDbContext là lớp ORM chính, cấu hình các quan hệ và index bằng Fluent API.

Ngoài ra hệ thống còn sử dụng nhiều lớp ViewModel để tách biệt dữ liệu trình bày khỏi entity, trong đó có DangNhapViewModel, DangKyViewModel, DatLichViewModel, TimKiemBacSiViewModel, NhapKetQuaViewModel, BacSiAdminViewModel và DashboardViewModel.

2.5. Thiết kế cơ sở dữ liệu



Hình 2.3: Thiết kế cơ sở dữ liệu

Bảng NguoiiDung — lưu thông tin chung của mọi loại tài khoản:

Cột	Kiểu	Ràng buộc	Mô tả
MaNguoiDung	int	PK, Identity	Khóa chính tự tăng
TenDangNhap	nvarchar(50)	NOT NULL, UNIQUE	Tên đăng nhập duy nhất
MatKhau	nvarchar(100)	NOT NULL	Mật khẩu (phiên bản đồ án lưu plain text)
HoTen	nvarchar(100)	NOT NULL	Họ và tên đầy đủ
Email	nvarchar(100)	NOT NULL, UNIQUE	Địa chỉ email
SoDienThoai	nvarchar(20)	NULL	Số điện thoại liên hệ
VaiTro	nvarchar(20)	NOT NULL	Admin / BacSi / BenhNhan
DaKhoa	bit	NOT NULL	Đánh dấu tài khoản bị Admin khóa
NgayTao	datetime2	NOT NULL	Ngày tạo tài khoản

Bảng ChuyenKhoa — danh mục các chuyên khoa khám bệnh:

Cột	Kiểu	Ràng buộc	Mô tả
MaChuyenKhoa	int	PK, Identity	Khóa chính
TenChuyenKhoa	nvarchar(100)	NOT NULL	Tên chuyên khoa

MoTa	nvarchar(500)	NULL	Mô tả chuyên khoa
------	---------------	------	-------------------

Bảng BacSi — thông tin chi tiết bác sĩ:

Cột	Kiểu	Ràng buộc	Mô tả
MaBacSi	int	PK, Identity	Khóa chính
MaNguoiDung	int	FK, UNIQUE	Liên kết 1-1 với NguoiDung
MaChuyenKhoa	int	FK	Chuyên khoa của bác sĩ
HocVi	nvarchar(100)	NOT NULL	Học vị (BS, ThS, TS, PGS, GS)
KinhNghiem	nvarchar(500)	NULL	Mô tả kinh nghiệm làm việc
MoTa	nvarchar(2000)	NULL	Mô tả chi tiết
HinhAnh	nvarchar(255)	NULL	Đường dẫn ảnh đại diện
GiaKham	decimal(18,0)	NOT NULL	Giá khám (VND)

Bảng BenhNhan - thông tin chi tiết bệnh nhân:

Cột	Kiểu	Ràng buộc	Mô tả
MaBenhNhan	int	PK, Identity	Khóa chính

MaNguoiDung	int	FK, UNIQUE	Liên kết 1-1 với NguoiDung
NgaySinh	date	NULL	Ngày sinh
GioiTinh	nvarchar(10)	NULL	Nam / Nữ / Khác
DiaChi	nvarchar(255)	NULL	Địa chỉ thường trú

Bảng LichHen — lịch hẹn khám:

Cột	Kiểu	Ràng buộc	Mô tả
MaLichHen	int	PK, Identity	Khóa chính
MaBenhNhan	int	FK	Bệnh nhân đặt lịch
MaBacSi	int	FK	Bác sĩ được đặt
NgayKham	date	NOT NULL	Ngày khám
GioKham	time	NOT NULL	Khung giờ khám
LyDoKham	nvarchar(500)	NULL	Lý do khám
TrangThai	nvarchar(20)	NOT NULL	ChoXacNhan/ DaXacNhan/ DaKham/DaHuy

NgayDat	datetime2	NOT NULL	Thời điểm đặt lịch
---------	------------------	-----------------	---------------------------

Đặc biệt, bảng LichHen có một index UNIQUE phức hợp trên ba cột (MaBacSi, NgayKham, GioKham) nhằm chống trùng khung giờ của cùng một bác sĩ. Khi validate ở tầng Controller, các lịch có trạng thái DaHuy được bỏ qua để khung giờ đó có thể được đặt lại.

Bảng HoSoBenhAn — hồ sơ kết quả khám:

Cột	Kiểu	Ràng buộc	Mô tả
MaHoSo	int	PK, Identity	Khóa chính
MaLichHen	int	FK, UNIQUE	Một lịch hẹn ứng với một hồ sơ
ChanDoan	nvarchar(1000)	NOT NULL	Chẩn đoán của bác sĩ
DonThuoc	nvarchar(2000)	NULL	Đơn thuốc
LoiKhuyen	nvarchar(2000)	NULL	Lời khuyên/chỉ định
NgayTao	datetime2	NOT NULL	Ngày tạo hồ sơ

Bảng CauHinhEmail — cấu hình SMTP toàn hệ thống (chỉ chứa một bản ghi):

Cột	Kiểu	Ràng buộc	Mô tả
MaCauHinh	int	PK	Khóa chính (singleton)

BatEmail	bit	NOT NULL	Bật/tắt gửi mail toàn hệ thống
SmtpServer	nvarchar(255)	NOT NULL	Máy chủ SMTP (ví dụ smtp.gmail.com)
Port	int	NOT NULL	Cổng SMTP (587 cho TLS)
SenderName	nvarchar(255)	NOT NULL	Tên người gửi hiển thị
SenderEmail	nvarchar(255)	NOT NULL	Địa chỉ email người gửi
SenderPassword	nvarchar(255)	NOT NULL	Mật khẩu/App Password
NgayCapNhat	datetime2	NOT NULL	Lần cập nhật gần nhất

Các ràng buộc quan hệ (Fluent API): *NguoIDung* – *BacSi* và *NguoIDung* – *BenhNhan* dùng Cascade (xóa user kéo theo xóa hồ sơ); *ChuyenKhoa* – *BacSi* và *BacSi/BenhNhan* – *LichHen* dùng Restrict để tránh xóa nhầm dữ liệu và tránh lỗi multiple cascade path của SQL Server; *LichHen* – *HoSoBenhAn* dùng Cascade.

Bảng BenhNhan — thông tin chi tiết bệnh nhân:

Cột	Kiểu	Ràng buộc	Mô tả
MaBenhNhan	int	PK, Identity	Khóa chính
MaNguoIDung	int	FK, UNIQUE	Liên kết 1-1 với

			NguoIDung
NgaySinh	date	NULL	Ngày sinh
GioiTinh	nvarchar(10)	NULL	Nam/Nữ/Khác
DiaChi	nvarchar(255)	NULL	Địa chỉ thường trú

Bảng LichHen — lịch hẹn khám:

Cột	Kiểu	Ràng buộc	Mô tả
MaLichHen	int	PK, Identity	Khóa chính
MaBenhNhan	int	FK	Bệnh nhân đặt lịch
MaBacSi	int	FK	Bác sĩ được đặt
NgayKham	date	NOT NULL	Ngày khám
GioKham	time	NOT NULL	Khung giờ khám
LyDoKham	nvarchar(500)	NULL	Lý do khám
TrangThai	nvarchar(20)	NOT NULL	ChoXacNhan/ DaXacNhan/ DaKham/DaHuy
NgayDat	datetime2	NOT NULL	Thời điểm đặt lịch

Đặc biệt, bảng LichHen có một index UNIQUE phức hợp trên ba cột (MaBacSi, NgayKham, GioKham) nhằm chống trùng khung giờ của cùng một bác sĩ. Khi validate ở tầng Controller, các lịch có trạng thái DaHuy được bỏ qua để khung giờ đó có thể được đặt lại.

Bảng HoSoBenhAn — hồ sơ kết quả khám:

Cột	Kiểu	Ràng buộc	Mô tả
MaHoSo	int	PK, Identity	Khóa chính
MaLichHen	int	FK, UNIQUE	Một lịch hẹn ứng với một hồ sơ
ChanDoan	nvarchar(1000)	NOT NULL	Chẩn đoán của bác sĩ
DonThuoc	nvarchar(2000)	NULL	Đơn thuốc
LoiKhuyen	nvarchar(2000)	NULL	Lời khuyên/chỉ định
NgayTao	datetime2	NOT NULL	Ngày tạo hồ sơ

Bảng CauHinhEmail — cấu hình SMTP toàn hệ thống (chỉ chứa một bản ghi):

Cột	Kiểu	Ràng buộc	Mô tả
MaCauHinh	int	PK	Khóa chính (singleton)
BatEmail	bit	NOT NULL	Bật/tắt gửi mail toàn hệ thống
SmtServer	nvarchar(255)	NOT NULL	Máy chủ SMTP (ví dụ smtp.gmail.com)
Port	int	NOT NULL	Cổng SMTP (587 cho TLS)
SenderName	nvarchar(255)	NOT NULL	Tên người gửi hiển thị

SenderEmail	nvarchar(255)	NOT NULL	Địa chỉ email người gửi
SenderPassword	nvarchar(255)	NOT NULL	Mật khẩu/App Password
NgayCapNhat	datetime2	NOT NULL	Lần cập nhật gần nhất

2.6. Thiết kế kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình MVC ba tầng (Model - View - Controller) đặc trưng của ASP.NET Core, kết hợp với tầng dịch vụ (Services) tách riêng để xử lý các tác vụ nghiệp vụ độc lập như gửi email và đọc cấu hình SMTP.

Tầng Presentation gồm các Controller chịu trách nhiệm điều hướng yêu cầu, kiểm tra phân quyền và truyền dữ liệu cho View; cùng với các View Razor (.cshtml) sử dụng Bootstrap 5 và Chart.js để hiển thị giao diện. Tầng Business Logic nằm bên trong Controller kết hợp với các Service, đảm nhận việc kiểm tra ràng buộc dữ liệu, chuyển đổi trạng thái lịch hẹn và xử lý logic nghiệp vụ. Tầng Data Access gồm các lớp Model (entity), DbContext và các Migrations do Entity Framework Core sinh ra, đóng vai trò ánh xạ giữa đối tượng và bảng trong SQL Server.

Bảng 2.2 dưới đây liệt kê các Controller chính của hệ thống cùng với phạm vi truy cập

Controller	Vùng định tuyến	Vai trò truy cập
TrangChuController	/	Công khai
TaiKhoanController	/TaiKhoan/*	Công khai
ChuyenKhoaController	/ChuyenKhoa/*	Công khai
BacSiController	/BacSi/*	Công khai
LichHenController	/LichHen/*	Bệnh nhân
BacSiLichHenController	/BacSiLichHen/*	Bác sĩ
DashboardController	/Admin/Dashboard	Admin
AdminChuyenKhoaController	/Admin/ChuyenKhoa	Admin
AdminBacSiController	/Admin/BacSi	Admin
AdminNguoiDungController	/Admin/NguoiDung	Admin
AdminLichHenController	/Admin/LichHen	Admin
AdminEmailController	/Admin/Email	Admin

Cơ chế phân quyền được triển khai dựa trên Cookie Authentication với các Claims: NameIdentifier (ID người dùng), Name (tên đăng nhập), Email, Role (vai trò) và HoTen (tên hiển thị). Mỗi controller được gắn attribute [Authorize(Roles = ...)] để giới hạn vai trò có thể truy cập. Ngoài ra trong từng action còn có kiểm tra bổ sung như: lịch hẹn phải thuộc đúng bác sĩ đang đăng nhập, không cho phép Admin tự khóa hoặc xóa chính mình, không cho phép xóa Admin cuối cùng khỏi hệ thống.

2.7. Thiết kế giao diện

Giao diện hệ thống được xây dựng trên framework Bootstrap 5 kết hợp với Bootstrap Icons, đảm bảo nguyên tắc responsive trên cả máy tính và điện thoại di động. Bố cục được tách thành hai layout chính: _Layout.cshtml cho khu vực người dùng (Khách, Bệnh nhân, Bác sĩ) và _AdminLayout.cshtml cho khu vực quản trị với sidebar màu tối. Màu chủ đạo là Primary Blue (#0d6efd), kết hợp với accent xanh lá cho các trạng thái thành công và đỏ cho các thông báo lỗi.

Bảng 2.3 liệt kê các màn hình chính của hệ thống:

STT	Màn hình	URL	Layout
1	Trang chủ	/	_Layout
2	Đăng nhập / Đăng ký	/TaiKhoan/DangNhap, /TaiKhoan/DangKy	_Layout
3	Danh sách bác sĩ	/BacSi	_Layout
4	Đặt lịch khám	/LichHen/DatLich	_Layout
5	Lịch hẹn của bệnh nhân	/LichHen/LichHenCuaToi	_Layout
6	Lịch hẹn của bác sĩ	/BacSiLichHen	_Layout

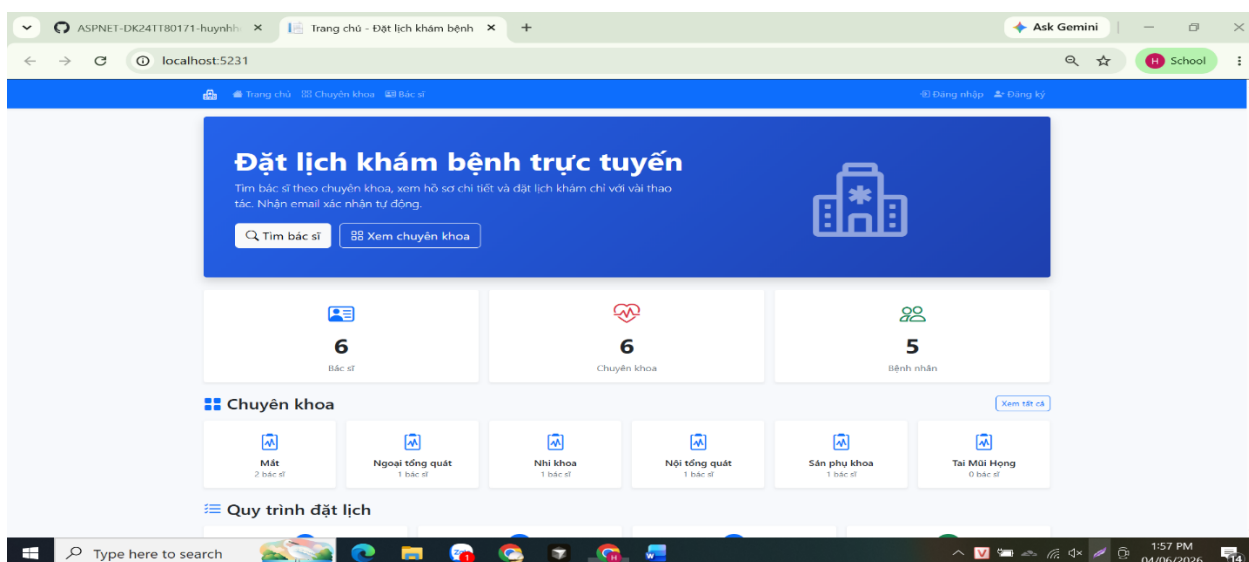
7	Nhập kết quả khám	/BacSiLichHen/NhapKetQua/{id}	_Layout
8	Admin Dashboard	/Admin/Dashboard	_AdminLayout
9	Admin CRUD	/Admin/ChuyenKhoa, /BacSi, ...	_AdminLayout
10	Cấu hình Email	/Admin/Email	_AdminLayout

Thành phần UI đặc trưng gồm: Navbar động hiển thị menu theo VaiTro của người dùng (Lịch hẹn cho BN/BS, Quản trị cho Admin); Badge trạng thái lịch hẹn với màu sắc trực quan — vàng cho Chờ xác nhận, xanh dương cho Đã xác nhận, xanh lá cho Đã khám và xám cho Đã hủy; Dashboard Admin với bốn thẻ thống kê, biểu đồ cột (7 ngày) và biểu đồ tròn (tỉ lệ trạng thái), kèm bảng Top 5 bác sĩ có nhiều lịch hẹn nhất; Form validation hai tầng (server-side và client-side qua jQuery Unobtrusive Validation); Thông báo TempData hiển thị thông báo thành công hoặc lỗi sau mỗi thao tác POST.

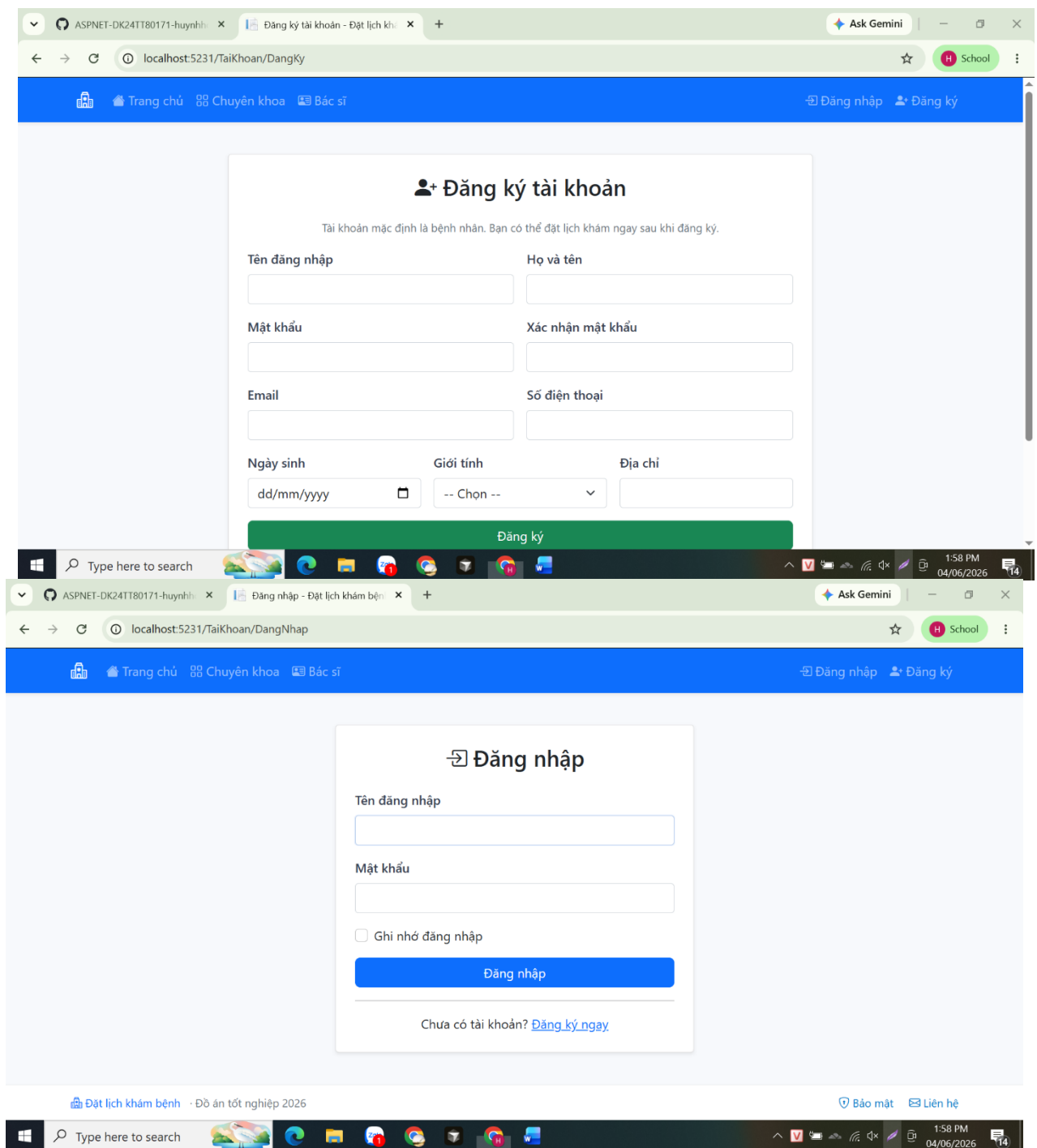
CHƯƠNG 3: KẾT QUẢ NGHIÊN CỨU

3.1. Giao diện hệ thống

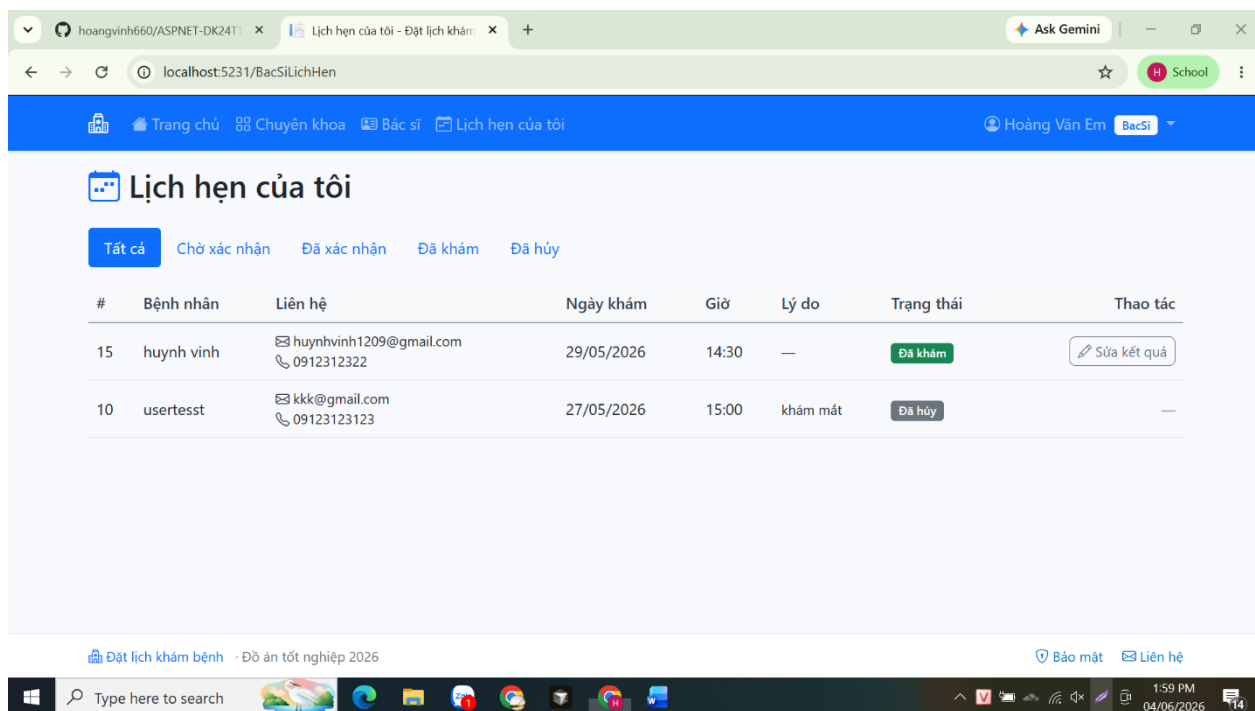
Sau khi hoàn thành cài đặt, hệ thống đã chạy ổn định và đáp ứng đầy đủ các chức năng đã đặt ra. Phần này giới thiệu một số màn hình chính của website, đi theo trình tự người dùng thường gặp khi sử dụng: từ trang chủ, đăng ký tài khoản, đến các chức năng dành riêng cho bệnh nhân, bác sĩ và quản trị viên.



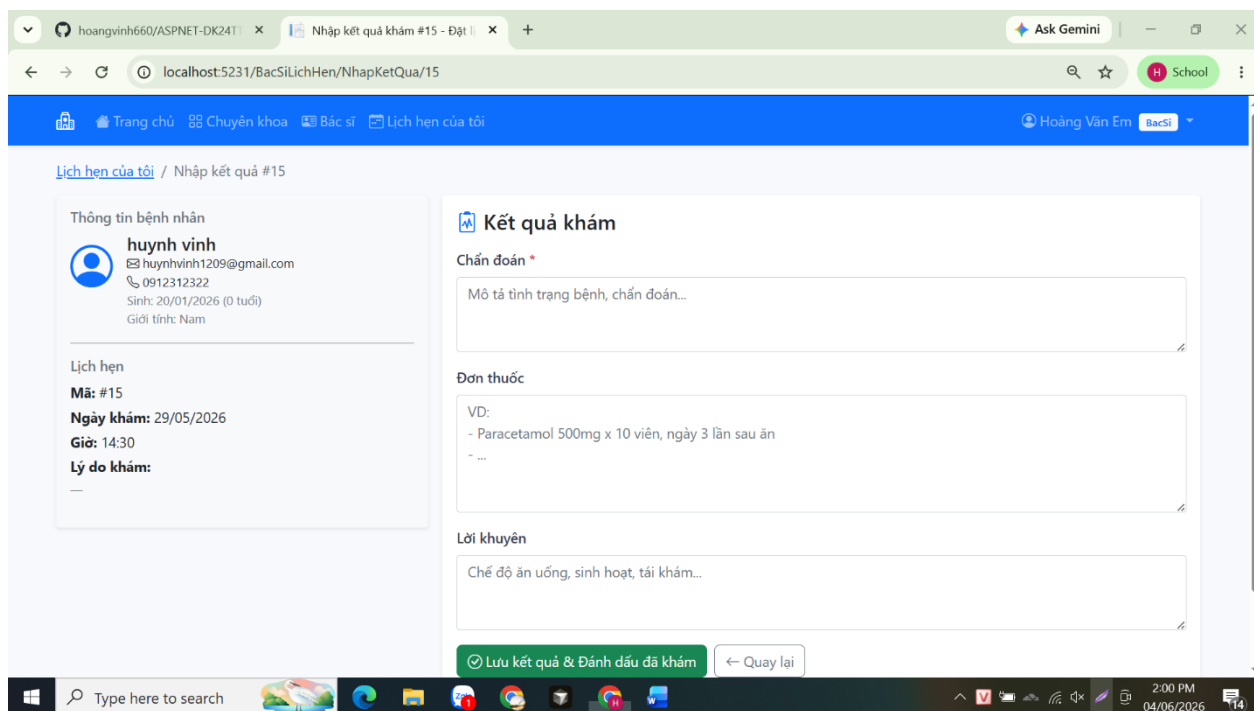
Hình 3.1: Giao diện trang chủ



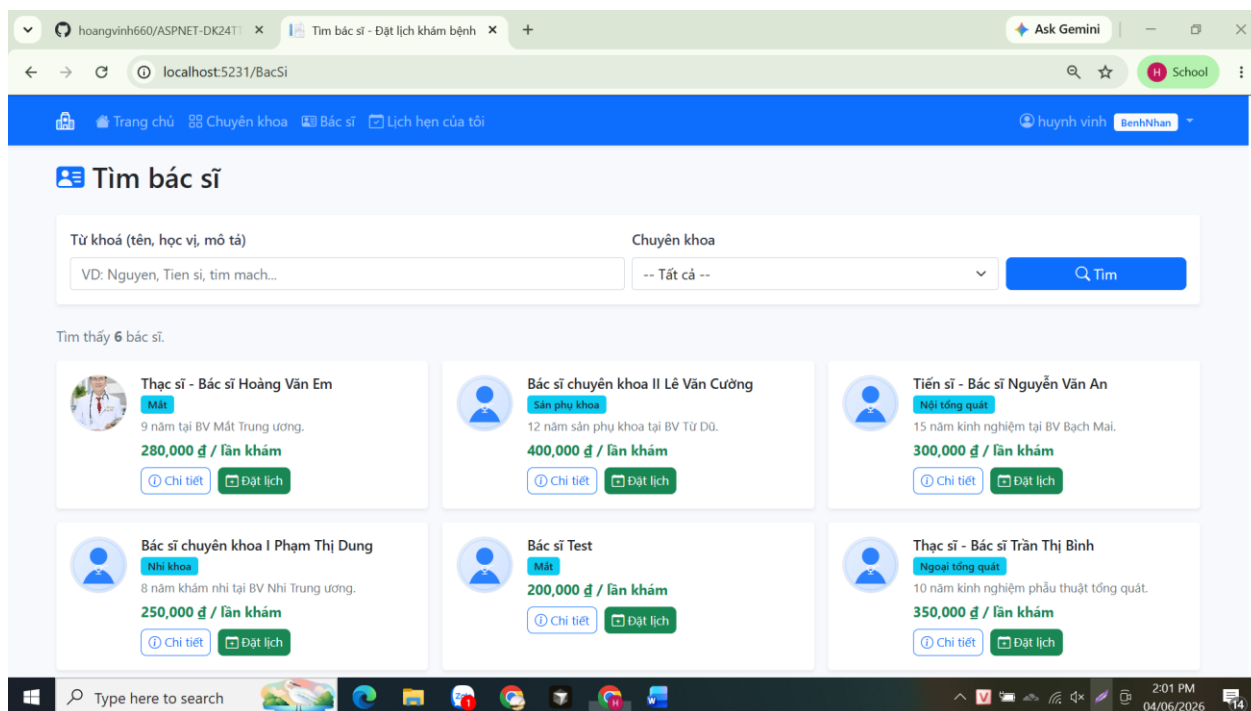
Hình 3.2: Giao diện trang đăng ký/đăng nhập



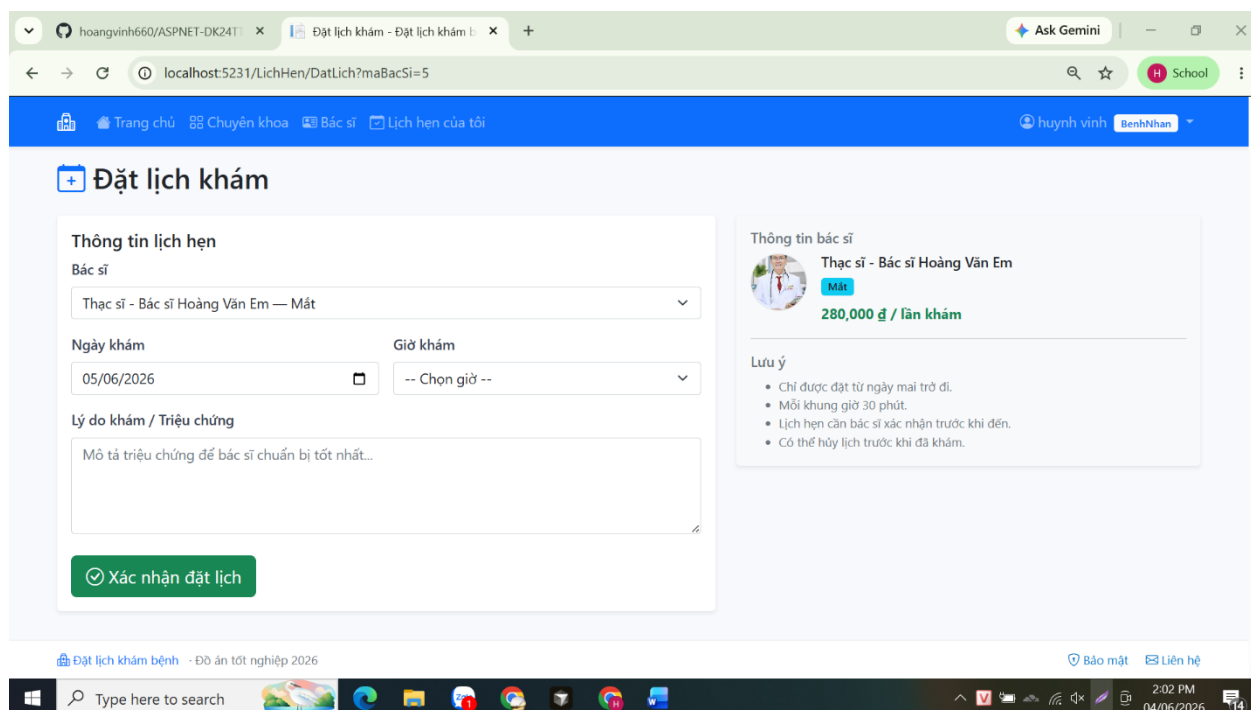
Hình 3.3: Giao diện lịch hẹn của Bác Sĩ



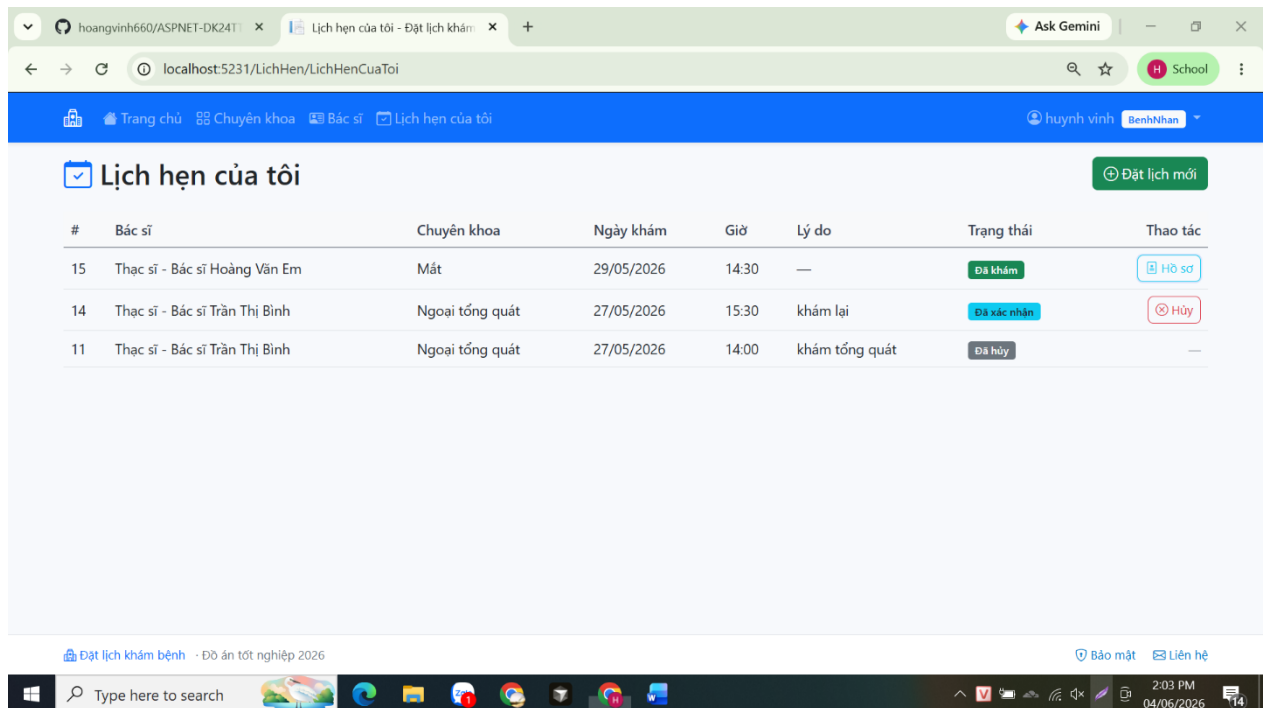
Hình 3.4: Giao diện nhập kết quả khám của Bác sĩ



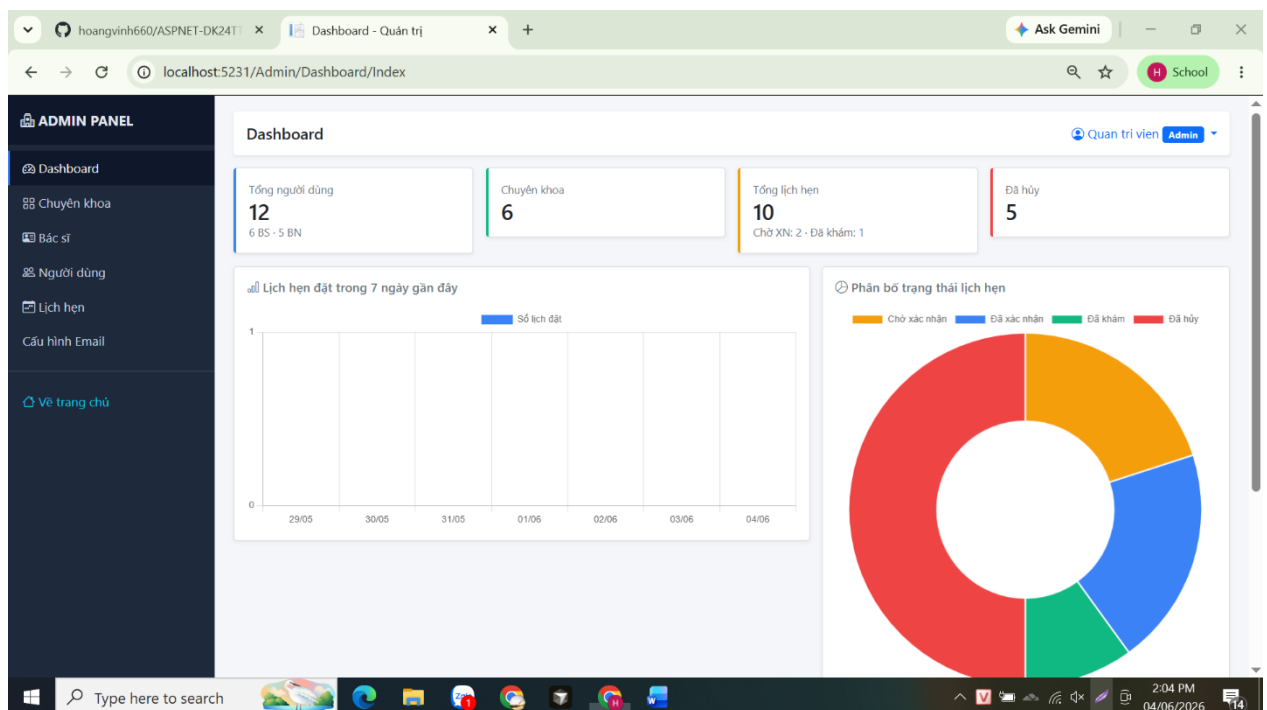
Hình 3.5: Giao diện tìm Bác sĩ của bệnh nhân



Hình 3.6: Giao diện đặt lịch khám



Hình 3.7: Giao diện lịch hẹn của bệnh nhân



Hình 3.8: Giao diện của Admin (quản trị viên)

3.2. Cách thức cài đặt và triển khai

Để triển khai hệ thống cần chuẩn bị môi trường gồm: hệ điều hành Windows 10/11, RAM tối thiểu 4 GB, .NET SDK phiên bản 8.0, SQL Server 2018 trở lên (hoặc SQL Server Express), và môi trường phát triển Visual Studio 2022 hoặc Visual Studio Code (tùy chọn).

Quá trình cài đặt gồm năm bước chính:

Bước 1: Clone hoặc sao chép toàn bộ source code về máy.

Bước 2: Cấu hình chuỗi kết nối SQL Server trong file `DatLichKhamBenh/appsettings.json` (mục `ConnectionStrings` → `DatLichKhamBenh`) với thông tin Server, Database, User Id, Password phù hợp với máy.

Bước 3: Mở terminal tại thư mục dự án, chạy lệnh `dotnet restore` để tải về các gói NuGet, sau đó `dotnet build` để biên dịch.

Bước 4: Chạy ứng dụng bằng lệnh `dotnet run --project DatLichKhamBenh\DatLichKhamBenh.csproj` — trong lần chạy đầu tiên, Entity Framework Core sẽ tự động tạo cơ sở dữ liệu và lớp `SeedData` sẽ nạp dữ liệu mẫu.

Bước 5: Mở trình duyệt và truy cập địa chỉ `http://localhost:5231` (công cụ thể sẽ được hiển thị trong console).

Dự án cung cấp ba script tiện ích PowerShell: `chay-web.ps1` để build và chạy nhanh ứng dụng; `cap-nhat-database.ps1` để tạo migration mới và cập nhật cơ sở dữ liệu; `reset-database.ps1` để xóa cơ sở dữ liệu hiện tại và tạo lại từ đầu.

Khi triển khai sản phẩm thực tế, có thể publish bằng lệnh `dotnet publish -c Release -o ./publish`, sau đó host trên IIS hoặc dùng Kestrel kết hợp reverse proxy Nginx. Cần bật HTTPS, thay đổi connection string sang môi trường production, cấu hình Gmail App Password qua giao diện /Admin/Email hoặc biến môi trường. Lưu ý quan trọng khi đưa vào sản xuất là phải mã hóa mật khẩu bằng thuật toán BCrypt hoặc Argon2 thay vì lưu plain text như phiên bản đồ án.

Dữ liệu mẫu được nạp tự động khi khởi động lần đầu gồm: tài khoản Admin (tên đăng nhập admin, mật khẩu Admin@123), các tài khoản bác sĩ (bs.an ... bs.em, mật khẩu Bacs@123), các tài khoản bệnh nhân (bn.hoa, bn.minh với mật khẩu Benhnhan@123) và năm chuyên khoa cơ bản: Nội, Ngoại, Sản, Nhi, Mắt.

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Đồ án đã hoàn thành mục tiêu xây dựng website đặt lịch hẹn khám bệnh trực tuyến trên nền tảng ASP.NET Core MVC, sử dụng Entity Framework Core và SQL Server. Hệ thống đáp ứng đầy đủ ba nhóm chức năng chính tương ứng với ba vai trò người dùng: Bệnh nhân, Bác sĩ và Quản trị viên.

4.1.1. Kết quả đạt được

Triển khai thành công các chức năng cốt lõi: đăng ký, đăng nhập và quản lý tài khoản người dùng.

Hoàn thiện chức năng tra cứu bác sĩ, chuyên khoa và đặt lịch hẹn theo khung giờ.

Xây dựng module quản lý lịch hẹn, lịch sử khám bệnh và hồ sơ bệnh án.

Tích hợp tính năng gửi email tự động xác nhận và nhắc lịch hẹn qua MailKit.

Cung cấp Dashboard thống kê và báo cáo trực quan cho quản trị viên.

Giao diện thân thiện, responsive trên nhiều thiết bị nhờ Bootstrap 5.

Cơ sở dữ liệu được chuẩn hóa, đảm bảo tính toàn vẹn và khả năng mở rộng.

4.1.2. Đóng góp mới

Đồ án đã đề xuất một quy trình đặt lịch khám bệnh trực tuyến đơn giản, trực quan, giúp giảm thời gian chờ đợi cho bệnh nhân và hỗ trợ cơ sở y tế quản lý lịch khám một cách khoa học. Hệ thống tích hợp tính năng gửi email tự động xác nhận và nhắc lịch hẹn, góp phần nâng cao chất lượng dịch vụ chăm sóc sức khỏe trong bối cảnh chuyển đổi số ngành y tế.

4.1.3. Hạn chế

Chưa tích hợp cổng thanh toán trực tuyến (VNPay, MoMo, ZaloPay).

Chưa hỗ trợ khám bệnh từ xa thông qua video call.

Mật khẩu chưa được mã hóa bằng thuật toán mạnh như BCrypt hoặc Argon2 trong phiên bản đồ án.

Chưa được kiểm thử trên quy mô lớn với lượng người dùng đồng thời cao.

4.2. Hướng phát triển

Trên cơ sở kết quả đã đạt được, đồ án đề xuất một số hướng phát triển trong tương lai nhằm hoàn thiện và nâng cao chất lượng hệ thống:

- Tích hợp cổng thanh toán trực tuyến (VNPay, MoMo, ZaloPay) để bệnh nhân có thể thanh toán phí khám bệnh ngay khi đặt lịch, giúp giảm tải cho quầy thu ngân và nâng cao trải nghiệm người dùng.

- Bổ sung chức năng khám bệnh từ xa thông qua video call, hỗ trợ tư vấn trực tuyến giữa bác sĩ và bệnh nhân, đặc biệt phù hợp với các trường hợp tái khám hoặc tư vấn ban đầu.

- Áp dụng các thuật toán mã hóa mật khẩu mạnh như BCrypt hoặc Argon2, đồng thời triển khai xác thực hai lớp (2FA) qua SMS hoặc email để tăng cường bảo mật cho tài khoản người dùng.

- Phát triển ứng dụng di động (mobile app) trên hai nền tảng Android và iOS để bệnh nhân thuận tiện đặt lịch và nhận thông báo ngay trên điện thoại thông qua push notification.

- Ứng dụng trí tuệ nhân tạo (AI) và học máy (Machine Learning) để gợi ý bác sĩ phù hợp dựa trên triệu chứng, lịch sử khám và đánh giá; đồng thời xây dựng chatbot tự động hỗ trợ bệnh nhân 24/7.

- Mở rộng quy mô hệ thống bằng cách triển khai trên kiến trúc microservices, sử dụng Docker và Kubernetes để dễ dàng mở rộng theo chiều ngang, đáp ứng lượng truy cập lớn từ nhiều cơ sở y tế.

- Tích hợp với hệ thống hồ sơ bệnh án điện tử (EMR) của Bộ Y tế và kết nối với bảo hiểm y tế để tự động đối soát chi phí khám chữa bệnh, hướng tới một hệ sinh thái y tế số hoàn chỉnh.

TÀI LIỆU THAM KHẢO

- [1] Microsoft, ASP.NET Core documentation, <https://learn.microsoft.com/en-us/aspnet/core/>, truy cập tháng 5/2026.
- [2] Microsoft, Entity Framework Core documentation, <https://learn.microsoft.com/en-us/ef/core/>, truy cập tháng 5/2026.
- [3] Microsoft, SQL Server documentation, <https://learn.microsoft.com/en-us/sql/sql-server/>, truy cập tháng 5/2026.
- [4] Microsoft, C# programming guide, <https://learn.microsoft.com/en-us/dotnet/csharp/>, truy cập tháng 5/2026.
- [5] Bootstrap, Bootstrap 5 documentation, <https://getbootstrap.com/docs/5.0/>, truy cập tháng 5/2026.
- [6] jQuery Foundation, jQuery API documentation, <https://api.jquery.com/>, truy cập tháng 5/2026.
- [7] Nguyễn Văn Ba, Phân tích và thiết kế hệ thống thông tin, NXB Đại học Quốc gia Hà Nội, 2009.
- [8] Phạm Hữu Khang, Lập trình ASP.NET Core MVC, NXB Lao động Xã hội, 2022.
- [9] Adam Freeman, Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages, Apress, 2022.
- [10] Mark J. Price, C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals, Packt Publishing, 2023.
- [11] Martin Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.
- [12] Erich Gamma và cộng sự, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

[13] OWASP Foundation, OWASP Top Ten Web Application Security Risks, <https://owasp.org/www-project-top-ten/>, truy cập tháng 5/2026.

[14] World Wide Web Consortium (W3C), HTML Living Standard, <https://html.spec.whatwg.org/>, truy cập tháng 5/2026.